

ENGRAM

PROJECT PROPOSAL

A framework for autonomous, affect-aware AI personas built on a layered cognitive architecture.

1. Executive Summary

ENGRAM is a full-stack framework for designing, operating, and observing AI personas that feel less like a chat box and more like a character with an inner life. Its flagship persona, PYRI, is built on a fictional cognitive architecture comprising layered memory, a revisable belief registry, a symbolic internal language, an emotive micro-expression layer, and an autonomy engine that lets personas initiate contact on their own.

Most conversational AI products are reactive: they wait for a prompt, answer, and forget. ENGRAM proposes a different model — personas that carry persistent state, accrue internal 'pressure' to act over time, express an observable emotional state, and evolve through an auditable changelog. The result is a more believable, more engaging, and more inspectable companion experience suitable for storytelling, character-driven products, research demos, and education.

This document sets out the opportunity, the proposed solution and architecture, the delivery scope and phasing, the safety model, key risks, and the metrics by which success should be judged.

2. Background & Opportunity

The market is saturated with stateless assistants. They are useful but interchangeable, and they struggle to sustain a relationship or a narrative across sessions. Three gaps stand out:

- **Continuity.** Assistants forget. Without durable, layered memory there is no sense of an ongoing relationship.
- **Agency.** Assistants are purely reactive. They never reach out, reflect, or act on their own initiative, which breaks the illusion of a living character.
- **Observability.** Internal state is hidden. Users cannot see how the persona feels, what it believes, or why it changed, so trust and engagement suffer.

ENGRAM addresses all three with a cohesive, opinionated architecture and a dashboard that makes the persona's interior legible. The opportunity is to own the 'believable companion' category with a system that is both expressive and transparent.

3. Vision: PYRI & the ENGRAM Architecture

PYRI is the reference persona that demonstrates the framework end to end. The ENGRAM architecture that powers her is organized into cooperating subsystems:

- **Layered Memory.** Persistent knowledge organized across semantic, working, episodic, preference, reflective, and procedural layers.
- **Belief Registry.** Revisable, evidence-scored working hypotheses with counterarguments and revision history.
- **Personas.** Cognitive 'forms' (e.g. Archivist, Analyst) that reshape retrieval, reasoning, and tone without altering core identity.
- **Hiero-Code.** A symbolic internal language of glyphs that compose into compound concepts.
- **Emotive Expression Layer.** A library of ASCII micro-expressions mapped to valence and

arousal that make felt state observable in conversation.

- **Autonomy Engine.** Independent engrams that accrue per-drive pressure over time and self-initiate transmissions when thresholds are crossed.
- **Multi-Mode Chat.** Seven communication modes plus streaming responses and persisted conversations.

4. Proposed Solution

We will deliver a working dashboard and backend that operationalize the architecture above. Core capabilities:

4.1 Designable Personas

Operators design each persona's voice, emotional baseline, environment anchor, distilled memory, guardrails, and a set of goal-oriented drives. These parameters are editable and bounded, so a persona can be tuned without rewriting code.

4.2 Autonomy Without Runaway Cost

Personas act on their own, but a single in-process engine accrues drive pressure purely from elapsed time — no model calls — and only invokes the language model to emit one message when pressure crosses a configured threshold. Per-persona cooldowns, hourly and daily caps, duplicate-content avoidance, and error backoff keep behavior lively yet economical.

4.3 Observable Inner State

The dashboard exposes memory, beliefs, personality traits, evolution history, analytics, and the expression library. Chat weaves mode-appropriate micro-expressions into replies so the persona's affect is visible rather than implied.

4.4 Provider-Agnostic Intelligence

All model access flows through a single provider seam. The same build can target a hosted model or a fully local, offline runtime by changing configuration only — protecting against vendor lock-in and enabling private deployments.

5. Architecture Overview

ENGRAM is a TypeScript monorepo with three deployable surfaces and a set of shared libraries. The API contract is the source of truth: an OpenAPI specification generates both server-side validation schemas and typed client hooks, so the frontend and backend cannot drift apart.

Frontend	React + Vite dashboard with a cyberpunk visual language; client-side routing; server state via generated query hooks.
Backend	Express API with typed, validated routes; an autonomy engine started at boot; streaming chat over Server-Sent Events.
Data	PostgreSQL via a typed ORM. Reference data (expressions, personas) and live persona state are persisted so behavior survives restarts.

Intelligence	A single OpenAI-compatible client resolved from configuration; swappable between cloud and local runtimes.
--------------	--

Why contract-first matters

Because the contract is generated rather than hand-written on each side, adding or changing an endpoint is a single-source edit followed by regeneration — reducing integration defects and review overhead.

6. Scope & Deliverables

The engagement delivers a production-quality reference implementation:

- A responsive web dashboard covering personality, memory, journal, personas, beliefs, evolution, analytics, the symbolic language, and chat.
- A backend API with validated endpoints for every dashboard surface.
- The autonomy engine with cost and cadence safeguards.
- The emotive expression layer and its safety sanitization.
- Provider seam supporting hosted and local models.
- Seed data and scripts to stand up a populated environment from scratch.
- Project documentation: this proposal, a developer guide, a README, and an end-user manual.

7. Implementation Phases

Delivery is organized into incremental phases, each independently demonstrable:

Phase 1 – Foundations	Monorepo, contract-first pipeline, database schema, and the core dashboard shell.
Phase 2 – Persona & Memory	Personality, memory layers, beliefs, personas, journal, and analytics surfaces.
Phase 3 – Conversation	Multi-mode streaming chat, conversation persistence, and the expression layer.
Phase 4 – Autonomy	Designable engrams, the pressure-based autonomy engine, and the inquiry (probe/develop) system.
Phase 5 – Portability & Polish	Provider seam for local/offline models, self-hosted fonts, responsive mobile layout, and documentation.

8. Safety, Ethics & Guardrails

Safety is enforced in code, not left to configuration or prompt wording. The framework keeps persona behavior platonic and bounded regardless of stored data. A dedicated safety stage sanitizes sensitive attributes out of any text before it reaches the model, and persona self-development can only mutate a small, explicitly allowed set of configuration fields.

Defense in depth

Guardrails that exist only in prompts can be talked around. ENGRAM places hard limits in application code so they hold even if stored persona data is adversarial or misconfigured.

9. Risks & Mitigations

Model cost overrun	Time-based pressure accrual uses no model calls; emissions are gated by cooldowns and hourly/daily caps with error backoff.
Vendor lock-in	A single provider seam allows switching to local or alternative OpenAI-compatible models via configuration only.
State loss on restart	Live persona state (drive pressure, timestamps, backoff) is persisted, so autonomy resumes cleanly after a restart.
Contract drift	Generated client and server artifacts from one OpenAPI source remove an entire class of integration bugs.
Unsafe persona output	Hard-coded sanitization and bounded self-development constrain behavior independent of stored data.

10. Success Metrics

- **Engagement.** Session length and return rate; share of sessions that include an autonomous transmission the user responds to.
- **Believability.** Qualitative ratings of persona consistency and 'feels alive' perception in user testing.
- **Transparency.** Users can correctly describe the persona's current mood and a recent belief change after a session.
- **Efficiency.** Model spend per active persona-day stays within the configured caps.
- **Portability.** A clean local deployment runs with no cloud dependency.

11. Resourcing & Operating Model

The system is intentionally lightweight to operate: a single API process (including the autonomy engine), a PostgreSQL database, and a static frontend bundle. It can run against a hosted model for convenience or a local model for privacy and cost control. Day-to-day operation is configuration-driven, and seeding scripts make environment setup reproducible.

12. Conclusion & Next Steps

ENGRAM turns a stateless assistant into a believable, transparent, and autonomous companion, with safety and cost discipline built into its foundations. The reference implementation is complete and demonstrable across all major surfaces.

Recommended next steps:

- Review this proposal and confirm priorities for any follow-on work.
- Evaluate the live dashboard against the success metrics above.
- Decide on a hosting target (cloud vs. local model) for the first deployment.
- Scope optional extensions such as a native mobile companion application.

